

PYTHON PRACTICE

Exercise 1 - Tuples

Create a tuple storing details of one employee:

(101, "Priya Mehta", "IT", 85000, "Pune")

Write code to:

1. Access name and salary using index.

2. Slice the tuple to get only the last 3 values.

3. Try to change the salary value.

Observe the error. Write a comment explaining why this happens.

4. Unpack all 5 values into separate variables.

Print each variable with a label.

5. Check if "IT" exists in the tuple.

6. Count how many times "Pune" appears in this tuple:

("Pune", "Mumbai", "Pune", "Delhi", "Pune")

7. Find the index of "Mumbai" in the above tuple.

1. Access name and salary using index.

```
t = (101, "Priya Mehta", "IT", 85000, "Pune")

# 1. Access name and salary using index.
print(f"Name: {t[1]} \nSalary: {t[3]}")
```

✓ 0.0s

Name: Priya Mehta

Salary: 85000

2. Slice the tuple to get only the last 3 values.

```
t = (101, "Priya Mehta", "IT", 85000, "Pune")

# 2. Slice the tuple to get only the last 3 values.
print(f"Last 3 values is: {t[-3:]}")
```

✓ 0.0s

Last 3 values is: ('IT', 85000, 'Pune')

3. Try to change the salary value.

Observe the error. Write a comment explaining

why this happens.

```
t = (101, "Priya Mehta", "IT", 85000, "Pune")

• # 3. Try to change the salary value.
# Observe the error. Write a comment explaining
# why this happens.
...
t[3] = 7000
print(t)
The above code show this error:
TypeError: 'tuple' object does not support item assignment

This code will raise an error because tuples are immutable in Python.
Tuple → immutable (cannot change)
...
try:
    t[3] = 75000
    print(t)
except TypeError:
    print("Tuples are immutable in Python.")
```

✓ 0.0s

Tuples are immutable in Python.

4. Unpack all 5 values into separate variables.

Print each variable with a label.

```
t = (101, "Priya Mehta", "IT", 85000, "Pune")

# 4. Unpack all 5 values into separate variables.
# Print each variable with a label.
emp_id, emp_name, department, salary, city = t
print(f"Emp_Id: {emp_id} \nEmp_Name: {emp_name} \nDepartment: {department} \nSalary: {salary} \nEmp_City: {city}")
```

✓ 0.0s Python

```
Emp_Id: 101
Emp_Name: Priya Mehta
Department: IT
Salary: 85000
Emp_City: Pune
```

5. Check if "IT" exists in the tuple.

```
t = (101, "Priya Mehta", "IT", 85000, "Pune")

# 5. Check if "IT" exists in the tuple.
if 'IT' in t:
    print("True")
else:
    print("False")
```

✓ 0.0s

True

6. Count how many times "Pune" appears in this tuple:

```
# 6. Count how many times "Pune" appears in this tuple:

t1 = ("Pune", "Mumbai", "Pune", "Delhi", "Pune")
count = 0
for i in t1:
    if i == 'Pune':
        count += 1
print(f"How many times '{i}' appears in this tuple is: '{count}'")
```

✓ 0.0s

How many times 'Pune' appears in this tuple is: '3'

7. Find the index of "Mumbai" in the above tuple.

```
# 7. Find the index of "Mumbai" in the above tuple.

t1 = ("Pune", "Mumbai", "Pune", "Delhi", "Pune")
print(f"Index of mumbai: {t1.index('Mumbai')}")
```

✓ 0.0s

Index of mumbai: 1

Exercise 2 - Sets

1. Find students present in both batches. (Intersection)

```
batch_a = {"Rahul", "Priya", "Amit", "Neha", "Karan"}
batch_b = {"Priya", "Karan", "Sneha", "Divya", "Rohan"}

# 1. Find students present in both batches. (Intersection)
print(f"Intersection: {batch_a & batch_b}")
```

✓ 0.0s

Intersection: {'Karan', 'Priya'}

2. Find all unique students across both batches. (Union)

```
batch_a = {"Rahul", "Priya", "Amit", "Neha", "Karan"}
batch_b = {"Priya", "Karan", "Sneha", "Divya", "Rohan"}

# 2. Find all unique students across both batches. (Union)
print(f"Uniq value: {batch_a | batch_b}")
```

✓ 0.0s

Uniq value: {'Rahul', 'Neha', 'Amit', 'Karan', 'Rohan', 'Sneha', 'Priya', 'Divya'}

3. Find students only in batch_a, not in batch_b. (Difference)

```
batch_a = {"Rahul", "Priya", "Amit", "Neha", "Karan"}
batch_b = {"Priya", "Karan", "Sneha", "Divya", "Rohan"}

# 3. Find students only in batch_a, not in batch_b. (Difference)
print(f"Difference: {batch_a - batch_b}")
```

✓ 0.0s

Difference: {'Neha', 'Rahul', 'Amit'}

4. Find students who are in one batch but not both. (Symmetric difference)

```
batch_a = {"Rahul", "Priya", "Amit", "Neha", "Karan"}
batch_b = {"Priya", "Karan", "Sneha", "Divya", "Rohan"}

# 4. Find students who are in one batch but not both. (Symmetric difference)
print(f"Symmetric Difference: {batch_a ^ batch_b}")
```

✓ 0.0s

Symmetric Difference: {'Divya', 'Amit', 'Rahul', 'Neha', 'Rohan', 'Sneha'}

5. Check if {"Priya", "Karan"} is a subset of batch_a.

```
batch_a = {"Rahul", "Priya", "Amit", "Neha", "Karan"}
batch_b = {"Priya", "Karan", "Sneha", "Divya", "Rohan"}

# 5. Check if {"Priya", "Karan"} is a subset of batch_a.
batch_c = {"Priya", "Karan"}
print(batch_c.issubset(batch_a))
```

✓ 0.0s

True

6. Add "Meera" to batch_a.

```
batch_a = {"Rahul", "Priya", "Amit", "Neha", "Karan"}
batch_b = {"Priya", "Karan", "Sneha", "Divya", "Rohan"}

# 6. Add "Meera" to batch_a.
batch_a.add("Meera")
print(f"After add method set is: {batch_a}")
```

✓ 0.0s

After add method set is: {'Rahul', 'Amit', 'Neha', 'Priya', 'Meera', 'Karan'}

7. Remove "Amit" from batch_a safely.

Use discard() so no error if name is missing.

```
batch_a = {"Rahul", "Priya", "Amit", "Neha", "Karan"}
batch_b = {"Priya", "Karan", "Sneha", "Divya", "Rohan"}

# 7. Remove "Amit" from batch_a safely.
# Use discard() so no error if name is missing.
batch_a.discard("Amit")
print(f"After discard method is: {batch_a}")
```

✓ 0.0s

After discard method is: {'Rahul', 'Neha', 'Priya', 'Karan'}

8. Convert batch_a to a sorted list and print it.

```
batch_a = {"Rahul", "Priya", "Amit", "Neha", "Karan"}
batch_b = {"Priya", "Karan", "Sneha", "Divya", "Rohan"}

# 8. Convert batch_a to a sorted list and print it.
print(f"After sorted method set is: {sorted(batch_a)}")
```

✓ 0.0s

After sorted method set is: ['Amit', 'Karan', 'Neha', 'Priya', 'Rahul']

Exercise 3 - Functions Basics

Write the following functions:

1. `greet(name)`

Returns "Hello, Priya! Welcome to Data Engineering."

Use f-string inside the function.

2. `calculate_tax(salary)`

If salary > 50000, tax is 20 percent.

If salary between 30000 and 50000, tax is 10 percent.

Below 30000, no tax.

Return the tax amount.

3. `is_palindrome(word)`

Returns True if the word reads the same forwards and backwards. Ignore case.

Test with: "Madam", "Python", "Racecar"

4. `count_vowels(sentence)`

Returns the count of vowels in the sentence.

Call all four functions and print results with clear labels.

1. `greet(name)`

Returns "Hello, Priya! Welcome to Data Engineering."

Use f-string inside the function.

```
# 1. greet(name)
# Returns "Hello, Priya! Welcome to Data Engineering."
# Use f-string inside the function.

def greet(name):
    print(f"Hello, {name} Welcome to Data Engineering.")

greet("Priya")
```

✓ 0.0s

Hello, Priya Welcome to Data Engineering.

```
# 2. calculate_tax(salary)

# If salary > 50000, tax is 20 percent.

# If salary between 30000 and 50000, tax is 10 percent.

# Below 30000, no tax.

# Return the tax amount.
```

```
# 2. calculate_tax(salary)
# If salary > 50000, tax is 20 percent.
# If salary between 30000 and 50000, tax is 10 percent.
# Below 30000, no tax.
# Return the tax amount.

def calculate_tax(salary):
    if salary > 50000:
        print(f"tax: {salary * 0.02}")
    elif 30000 <= salary <= 50000:
        print(f"tax: {salary * 0.1}")
    else:
        print("No tax")

calculate_tax(25000)
```

✓ 0.0s

No tax

```
# 3. is_palindrome(word)

# Returns True if the word reads the same forwards
# and backwards. Ignore case.

# Test with: "Madam", "Python", "Racecar"
```

```
# 3. is_palindrome(word)
# Returns True if the word reads the same forwards
# and backwards. Ignore case.
# Test with: "Madam", "Python", "Racecar"

def is_palindrome(word):
    word = word.lower()
    if word == word[::-1]:
        return True
    else:
        return False

is_palindrome("Madam")

def is_palindrome1(word):
    word = word.lower()
    return word == word[::-1]

print(is_palindrome1("madam"))
print(is_palindrome1("PYTHON"))
print(is_palindrome1("Recear"))
```

✓ 0.0s

True
False
False

4. count_vowels(sentence)

Returns the count of vowels in the sentence.

```
# 4. count_vowels(sentence)
# Returns the count of vowels in the sentence.
def count_vowels(sentence):
    count = 0
    vowels = ['a', 'e', 'i', 'o', 'u']
    for i in sentence:
        if i in vowels:
            count += 1
    print(f"Count of vowels is: {count}")

count_vowels("python is easy language.")
```

✓ 0.0s

Count of vowels is: 8

Exercise 4 - Functions with Default Arguments

Write a function called employee_summary that takes:

- name (required)
- department (required)
- salary (default value: 50000)
- city (default value: "Pune")

The function prints a formatted employee card.

Call the function four times:

1. Passing all four arguments.
2. Passing only name and department.
3. Passing name, department, and a custom salary.
4. Passing name, department, and a custom city.

Observe how default arguments work in each case.

Concepts: def, default arguments, f-string


```
# Write a function called employee_summary that takes:
# - name (required)
# - department (required)
# - salary (default value: 50000)
# - city (default value: "Pune")
#
# The function prints a formatted employee card.

def employee_summary(name, department, salary=50000, city='Pune'):
    print(f"Name: {name} \nDepartment: {department} \nSalary: {salary} \nCity: {city}")

employee_summary("Amit", "IT")
```

✓ 0.0s

Name: Amit
Department: IT
Salary: 50000
City: Pune

1. Passing all four arguments.

```
# 1. Passing all four arguments.
employee_summary("Amit", "IT", 75000, "Kolhapur")
```

✓ 0.0s

Name: Amit
Department: IT
Salary: 75000
City: Kolhapur

2. Passing only name and department.

```
1
# 2. Passing only name and department.
employee_summary("Rahul", "HR")
```

✓ 0.0s

Name: Rahul
Department: HR
Salary: 50000
City: Pune

3. Passing name, department, and a custom salary.

4. Passing name, department, and a custom city.

```
def employee_summary(name, department, salary=50000, city='Pune'):
    print(f"Name: {name} \nDepartment: {department} \nSalary: {salary} \nCity: {city}")

# 3. Passing name, department, and a custom salary.
employee_summary("Karan", "Finance", 90000)

print("====="*5)

# 4. Passing name, department, and a custom city.
employee_summary("Vishal", "HR", city="Jalna")
```

✓ 0.0s

```
Name: Karan
Department: Finance
Salary: 90000
City: Pune
=====
Name: Vishal
Department: HR
Salary: 50000
City: Jalna
```

Exercise 5 - *args and **kwargs

Part A - *args

Write a function called `total_sales(*args)` that:

- Accepts any number of sales figures
- Returns the total and average

Test with:

`total_sales(1500, 2300, 4100)`

`total_sales(800, 1200, 3300, 2700, 4500)`

Part A - *args

Write a function called `total_sales(*args)` that:

- # - Accepts any number of sales figures
- # - Returns the total and average

```
# Part A - *args
|
# Write a function called total_sales(*args) that:
# - Accepts any number of sales figures
# - Returns the total and average

def total_sales(*args):
    print(f"Total Sum: {sum(args)}")
    print(f"Average: {sum(args) / len(args)}")

total_sales(1500, 2300, 4100)
total_sales(800, 1200, 3300, 2700, 4500)
```

✓ 0.0s

```
Total Sum: 7900
Average: 2633.3333333333335
Total Sum: 12500
Average: 2500.0
```

Part B - **kwargs

Write a function called `create_profile(**kwargs)` that:

- # - Accepts any number of keyword arguments
- # - Prints each key and value on a new line

Test with:

```
# create_profile(name="Rahul", city="Pune",
#               role="Data Engineer", salary=72000)
```

```

# Part B - **kwargs
|
# Write a function called create_profile(**kwargs) that:
# - Accepts any number of keyword arguments
# - Prints each key and value on a new line

def create_profile(**kwargs):
    for key, value in kwargs.items():
        print(key, ': ', value)

create_profile(name="Rahul", city="Pune", role="Data Engineer", salary=72000)

```

✓ 0.0s

```

name : Rahul
city : Pune
role : Data Engineer
salary : 72000

```

Part C - Combined

```

# Write a function called log_event(event_type,
# *args, **kwargs) that:
# - Prints the event type
# - Prints all positional values as a list
# - Prints all keyword values as key:value pairs

```

```

# Part C - Combined
#
# Write a function called log_event(event_type,
# *args, **kwargs) that:
# - Prints the event type
# - Prints all positional values as a list
# - Prints all keyword values as key:value pairs
#
# Test with:
# log_event("pipeline_run", "step1", "step2",
#           status="success", rows=5000)

def log_event(event_type, *args, **kwargs):
    print(f"Events type: {event_type}")
    print(f"Values: {list(args)}")

    for key, value in kwargs.items():
        print(key, ': ', value)

log_event("pipeline_run", "step1", "step2", status="success", rows=5000)

```

✓ 0.0s

```

Events type: pipeline_run
Values: ['step1', 'step2']
status : success
rows : 5000

```

Exercise 6 - List Comprehensions

Given this list of salaries:

```
salaries = [72000, 48000, 85000, 61000, 55000,  
            52000, 47000, 91000, 38000, 67000]
```

Using list comprehensions (one line each):

1. Create a list of salaries above 60000.
2. Create a list of salaries after applying
10 percent increment to each.
3. Create a list of "High" or "Low" for each salary
based on whether it is above 60000 or not.
4. Create a list of squares of all salaries
below 50000.

Then given this list of names:

```
names = ["rahul", "AMIT", "Priya", "NEHA", "karan"]
```

5. Create a new list with all names in Title Case.
6. Create a new list of names that have
more than 4 characters.

- # 1. Create a list of salaries above 60000.
- # 2. Create a list of salaries after applying
- # 10 percent increment to each.
- # 3. Create a list of "High" or "Low" for each salary
- # based on whether it is above 60000 or not.
- # 4. Create a list of squares of all salaries
- # below 50000.

```
salaries = [72000, 48000, 85000, 61000, 55000,
            52000, 47000, 91000, 38000, 67000]

# 1. Create a list of salaries above 60000.
above_60000_salary = [i for i in salaries if i > 60000]
print(f"Salary above 60000 list is: {above_60000_salary}")

# 2. Create a list of salaries after applying
# 10 percent increment to each.
increment = [round(i*1.10) for i in salaries]
print(f"After 10 percent increment list is: {increment}")

# 3. Create a list of "High" or "Low" for each salary
# based on whether it is above 60000 or not.
salary = ["High" if i > 60000 else "Low" for i in salaries]
print(salary)

# 4. Create a list of squares of all salaries
# below 50000.

squares = [i**2 for i in salaries if i < 50000]
print(f"Square: {squares}")
```

✓ 0.0s

Python

Salary above 60000 list is: [72000, 85000, 61000, 91000, 67000]
 After 10 percent increment list is: [79200, 52800, 93500, 67100, 60500, 57200, 51700, 100100, 41800, 73700]
 ['High', 'Low', 'High', 'High', 'Low', 'Low', 'Low', 'High', 'Low', 'High']
 Square: [2304000000, 2209000000, 1444000000]

- # 5. Create a new list with all names in Title Case.
- # 6. Create a new list of names that have

```
# Then given this list of names:
names = ["rahul", "AMIT", "Priya", "NEHA", "karan"]

# 5. Create a new list with all names in Title Case.
title_case = [i.title() for i in names]
print(f"Title case: {title_case}")

# 6. Create a new list of names that have
# more than 4 characters.
only_4_characters_list = [char for char in names if len(char) > 4]
print(f"Only more than 4 characters list is: {only_4_characters_list}")
```

✓ 0.0s

Title case: ['Rahul', 'Amit', 'Priya', 'Neha', 'Karan']
 Only more than 4 characters list is: ['rahul', 'Priya', 'karan']

Exercise 7 - Dict Comprehensions

Part A

Given two lists:

Create a dictionary {name: salary} using

dict comprehension.

```
# Part A
# Given two lists:
# Create a dictionary {name: salary} using
# dict comprehension.
names = ["Rahul", "Priya", "Amit", "Neha", "Karan"]
salaries = [72000, 85000, 48000, 61000, 55000]

dictionary = { Name : Salary for Name, Salary in zip(names,salaries)}
print(f"Dictionary: {dictionary}")
```

✓ 0.0s

Dictionary: {'Rahul': 72000, 'Priya': 85000, 'Amit': 48000, 'Neha': 61000, 'Karan': 55000}

Part B

From the dictionary created above:

1. Create a new dict with only employees
earning above 60000.
2. Create a new dict where each salary
is increased by 5000.
3. Create a new dict where keys are names
and values are "High" or "Low" based
on salary threshold of 60000.

```
# Part B
# From the dictionary created above:
# 1. Create a new dict with only employees earning above 60000.
above_60000_salary = {Name: Salary for Name, Salary in zip(names, salaries) if Salary > 60000}
print(f"Only This Employees Earning Above 60000 dict is: {above_60000_salary}")

print("====="*9)
# 2. Create a new dict where each salary is increased by 5000.
New_Dict = {Name: Salary + 5000 for Name, Salary in zip(names,salaries)}
print(f"Increased 5000 salary dic is: {New_Dict}")

print("====="*9)
# 3. Create a new dict where keys are names
# and values are "High" or "Low" based
# on salary threshold of 60000.
New_Dict1 = {Name: "High" if Salary > 60000 else "Low" for Name, Salary in zip(names,salaries)}
print(New_Dict1)
```

✓ 0.0s Python

Only This Employees Earning Above 60000 dict is: {'Rahul': 72000, 'Priya': 85000, 'Neha': 61000}
=====

Increased 5000 salary dic is: {'Rahul': 77000, 'Priya': 90000, 'Amit': 53000, 'Neha': 66000, 'Karan': 60000}
=====

{'Rahul': 'High', 'Priya': 'High', 'Amit': 'Low', 'Neha': 'High', 'Karan': 'Low'}

SQL PRACTICE

SQL Exercise 1 - Aggregate Basics

1. Count total number of orders.
2. Find total revenue across all orders.
 $\text{total_amount} = \text{quantity} \times \text{unit_price}$
3. Find average order value.
4. Find the most expensive unit price.
5. Find the cheapest unit price.
6. Count how many distinct cities placed orders.
7. Count how many distinct products were ordered.

```
-- 1. Count total number of orders.
SELECT COUNT(*) AS ORDERS_COUNT FROM ORDERS2;

-- 2. Find total revenue across all orders.
-- total_amount = quantity x unit_price
SELECT SUM(QUANTITY * UNIT_PRICE) AS TOTAL_REVENUE FROM ORDERS2;

-- 3. Find average order value.
SELECT AVG(QUANTITY * UNIT_PRICE) AS AVERAGE_VALUE FROM ORDERS2;

-- 4. Find the most expensive unit price.
SELECT MAX(UNIT_PRICE) AS EXPENSIVE_UNIT_PRICE FROM ORDERS2;

-- 5. Find the cheapest unit price.
SELECT MIN(UNIT_PRICE) AS CHEAPEST_UNIT_PRICE FROM ORDERS2;

-- 6. Count how many distinct cities placed orders.
SELECT COUNT(DISTINCT CITY) AS COUNT FROM ORDERS2;

-- 7. Count how many distinct products were ordered.
SELECT COUNT(DISTINCT PRODUCT) AS COUNT_PRODUCT FROM ORDERS2;
SELECT * FROM ORDERS2;
```


SQL Exercise 2 - GROUP BY Revision

1. Total revenue per category.
2. Number of orders per city.
3. Average quantity ordered per product.
4. Total revenue per city, ordered highest first.
5. Number of orders per customer.

(Every customer here has one order,
but write the query correctly anyway.)

```
-- 1. Total revenue per category.
SELECT CATEGORY, SUM(QUANTITY * UNIT_PRICE) AS TOTAL_REVENUE FROM ORDERS2 GROUP BY CATEGORY;

-- 2. Number of orders per city.
SELECT CITY, COUNT(*) AS COUNT_ORDER_PER_CITY FROM ORDERS2 GROUP BY CITY;

-- 3. Average quantity ordered per product.
SELECT PRODUCT, AVG(QUANTITY * UNIT_PRICE) AS AVERAGE FROM ORDERS2 GROUP BY PRODUCT;

-- 4. Total revenue per city, ordered highest first.
SELECT CITY, SUM(QUANTITY * UNIT_PRICE) AS TOTAL_REVENUE FROM ORDERS2 GROUP BY CITY ORDER BY TOTAL_REVENUE DE

-- 5. Number of orders per customer.
-- (Every customer here has one order,
-- but write the query correctly anyway.)
SELECT CUSTOMER_NAME, COUNT(*) AS COUNT FROM ORDERS2 GROUP BY CUSTOMER_NAME;
```

SQL Exercise 3 - WHERE with GROUP BY

1. Total revenue per category,
only for orders from Pune.
2. Count of orders per city,
only for Electronics category.
3. Average unit price per category,
only for orders where quantity is more than 1.

```
-- SQL Exercise 3 - WHERE with GROUP BY

-- 1. Total revenue per category,
-- only for orders from Pune.
SELECT CATEGORY, SUM(QUANTITY * UNIT_PRICE) AS TOTAL_REVENUE FROM ORDERS2 WHERE CITY = 'Pune' GROUP BY CATEGORY;
SELECT * FROM ORDERS2;

-- 2. Count of orders per city,
-- only for Electronics category.
SELECT CITY, COUNT(*) AS COUNT_OF_ORDER FROM ORDERS2 WHERE CATEGORY = 'Electronics' GROUP BY CITY;

-- 3. Average unit price per category,
-- only for orders where quantity is more than 1.
SELECT CATEGORY, AVG(UNIT_PRICE) AS AVERAGE FROM ORDERS2 WHERE QUANTITY > 1 GROUP BY CATEGORY;
```

SQL Exercise 4 - HAVING Revision

1. Show cities that have placed more than 1 order.
2. Show categories where total revenue exceeds 100000.
3. Show products where average quantity ordered is more than 1.
4. Show cities where total quantity ordered is more than 3.

```
-- SQL Exercise 4 - HAVING Revision
--
-- 1. Show cities that have placed more than 1 order.
SELECT CITY, COUNT(*) FROM ORDERS2 GROUP BY CITY HAVING COUNT(*) > 1;
SELECT * FROM ORDERS2;

-- 2. Show categories where total revenue
-- exceeds 100000.
SELECT CATEGORY, SUM(QUANTITY * UNIT_PRICE) AS TOTAL_REVENUE FROM ORDERS2 GROUP BY CATEGORY HAVING SUM(QUANTITY * UNIT_PRICE) > 100000;

-- 3. Show products where average quantity ordered
-- is more than 1.
SELECT PRODUCT, AVG(QUANTITY) AS AVERAGE FROM ORDERS2 GROUP BY PRODUCT HAVING AVG(QUANTITY) > 1;

-- 4. Show cities where total quantity ordered
-- is more than 3.
SELECT CITY, SUM(QUANTITY) AS TOTAL_QUANTITY FROM ORDERS2 GROUP BY CITY HAVING SUM(QUANTITY) > 3;
```

Python

SQL Exercise 5 - Combined Challenge

Write a single query that produces this summary:

- Category name
- Total orders
- Total quantity sold
- Total revenue
- Average order value (rounded to 0 decimals)

Show only categories with total revenue above 50000.

Order by total revenue descending.

```
SELECT CATEGORY,  
COUNT(*) AS TOTAL_ORDER,  
SUM(QUANTITY) AS TOTAL_QUANTITY,  
SUM(QUANTITY * UNIT_PRICE) AS TOTAL_REVENUE,  
ROUND(AVG(QUANTITY * UNIT_PRICE), 0) AS AVERAGE  
FROM ORDERS2  
GROUP BY CATEGORY  
HAVING SUM(QUANTITY * UNIT_PRICE) > 50000  
ORDER BY AVERAGE DESC;
```

LINUX PRACTICE

Linux Task 1 - tar (Archive)

Setup: Create a folder called project_files.

Inside it create 4 files:

employees.csv orders.csv summary.txt notes.txt

Now perform:

1. Create a tar archive of the entire folder.

Output file: project_files.tar

2. List all files inside the archive

without extracting.

3. Extract the archive into a new folder

called extracted_files.

4. Create a compressed tar archive using gzip.

Output file: project_files.tar.gz

5. Extract the .tar.gz archive.

6. Check and compare sizes of:

- Original folder

- .tar file

- .tar.gz file

Use: du -sh

1. Create a tar archive of the entire folder.

Output file: project_files.tar

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ tar -cvf project_files.tar project_files/
project_files/
project_files/employees.csv
project_files/notes.txt
project_files/orders.csv
project_files/summary.txt
```

2. List all files inside the archive without extracting

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ tar -tvf project_files.tar
drwxrwxrwx prashant/prashant 0 2026-03-18 09:29 project_files/
-rwxrwxrwx prashant/prashant 0 2026-03-18 09:29 project_files/employees.csv
-rwxrwxrwx prashant/prashant 0 2026-03-18 09:29 project_files/notes.txt
-rwxrwxrwx prashant/prashant 0 2026-03-18 09:29 project_files/orders.csv
-rwxrwxrwx prashant/prashant 0 2026-03-18 09:29 project_files/summary.txt
```

3. Extract the archive into a new folder called extracted_files.

```
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/linux$ mkdir -p extracted_files
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/linux$ tar -xvf project_files.tar -C extracted_files/
project_files/
project_files/employees.csv
tar: project_files/employees.csv: time stamp 2026-03-18 09:29:11 is 47673.211965061 s in the future
project_files/notes.txt
tar: project_files/notes.txt: time stamp 2026-03-18 09:29:11 is 47673.207074784 s in the future
project_files/orders.csv
tar: project_files/orders.csv: time stamp 2026-03-18 09:29:11 is 47673.202834067 s in the future
project_files/summary.txt
tar: project_files/summary.txt: time stamp 2026-03-18 09:29:11 is 47673.199160994 s in the future
tar: project_files: time stamp 2026-03-18 09:29:11 is 47673.197089093 s in the future
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/linux$ ls extracted_files/
project_files
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/linux$
```

4. Create a compressed tar archive using gzip.

Output file: project_files.tar.gz

```
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/linux$ tar -czvf project_files.tar.gz project_files
project_files/
project_files/employees.csv
project_files/notes.txt
project_files/orders.csv
project_files/summary.txt
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/linux$ ls -rlth
total 5.6M
-rwxrwxrwx 1 prashant prashant 62K Sep  8 2025 KODE-X_LINUX_DE.html
-rwxrwxrwx 1 prashant prashant 824 Sep  9 2025 '9sep25(1).txt'
-rwxrwxrwx 1 prashant prashant 1.9K Sep 10 2025 '10sep25(0).txt'
-rwxrwxrwx 1 prashant prashant 4.8K Sep 11 2025 '10sep25(1).txt'
-rwxrwxrwx 1 prashant prashant 23K Sep 11 2025 Data_Projects.txt
-rwxrwxrwx 1 prashant prashant 6.6K Sep 12 2025 basic_project.txt
-rwxrwxrwx 1 prashant prashant 1.8K Sep 17 10:54 'pratice que for loop.txt'
-rwxrwxrwx 1 prashant prashant 163 Sep 22 13:30 '8sep25(1).txt'
-rwxrwxrwx 1 prashant prashant 2.1K Sep 26 11:40 'Exercise 3.txt'
-rwxrwxrwx 1 prashant prashant 3.8K Sep 27 21:13 'du,df command.txt'
-rwxrwxrwx 1 prashant prashant 7.2K Oct  3 16:59 'All Commands 8sep25.txt'
-rwxrwxrwx 1 prashant prashant 17K Oct  7 15:23 'loop all task.txt'
-rwxrwxrwx 1 prashant prashant 5.4M Nov 11 21:59 how-linux-works-what-every-superuser-should-know-3rd-edition.pdf
-rwxrwxrwx 1 prashant prashant 4.0K Jan  9 12:19 loop
-rwxrwxrwx 1 prashant prashant 4.0K Jan  9 12:36 loop
-rwxrwxrwx 1 prashant prashant 1.4K Jan  9 12:41 complete_linux_note.ipynb
-rwxrwxrwx 1 prashant prashant 6.0K Mar  8 17:32 Exercise_data.bash
-rwxrwxrwx 1 prashant prashant 229 Mar 17 11:14 employees.csv
-rwxrwxrwx 1 prashant prashant 4.0K Mar 18 2026 project_files
-rwxrwxrwx 1 prashant prashant 10K Mar 18 2026 project_files.tar
-rwxrwxrwx 1 prashant prashant 4.0K Mar 18 2026 extracted_files
-rwxrwxrwx 1 prashant prashant 208 Mar 18 2026 project_files.tar.gz
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/linux$
```

5. Extract the .tar.gz archive.

```
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/linux$ tar -xvzf project_files.tar.gz
project_files/
project_files/employees.csv
tar: project_files/employees.csv: time stamp 2026-03-18 09:29:11 is 47361.444105244 s in the future
project_files/notes.txt
tar: project_files/notes.txt: time stamp 2026-03-18 09:29:11 is 47361.438069474 s in the future
project_files/orders.csv
tar: project_files/orders.csv: time stamp 2026-03-18 09:29:11 is 47361.430293497 s in the future
project_files/summary.txt
tar: project_files/summary.txt: time stamp 2026-03-18 09:29:11 is 47361.42410694 s in the future
tar: project_files: time stamp 2026-03-18 09:29:11 is 47361.422414694 s in the future
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/linux$
```

6. Check and compare sizes of:

- Original folder
- .tar file
- .tar.gz file

Use: du -sh

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ du -sh project_files.tar
12K    project_files.tar
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ du -sh project_files.tar.gz
0      project_files.tar.gz
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux$ |
```

Linux Task 2 - gzip and zip

1. Compress employees.csv using gzip.

Observe that the original file is replaced.

2. Decompress it using gunzip.

3. Compress employees.csv using gzip but keep the original file intact.

Flag: gzip -k

4. Create a zip archive containing employees.csv and orders.csv.

Output: data_files.zip

5. List contents of the zip file.

6. Extract the zip file into a folder called unzipped_data.

Commands: gzip gunzip gzip -k, zip unzip -l unzip -d

1. Compress employees.csv using gzip.

Observe that the original file is replaced.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ gzip employees.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ ls -lrth
total 0
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 summary.txt
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 orders.csv
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 notes.txt
-rwxrwxrwx 1 prashant prashant 34 Mar 18 2026 employees.csv.gz
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ |
```

2. Decompress it using gunzip.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ gunzip employees.csv.gz
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ ls -lrth
total 0
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 summary.txt
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 orders.csv
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 notes.txt
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 employees.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ |
```

3. Compress employees.csv using gzip but

keep the original file intact.

Flag: gzip -k

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ gzip -k employees.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ ls -lrth
total 0
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 summary.txt
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 orders.csv
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 notes.txt
-rwxrwxrwx 1 prashant prashant 34 Mar 18 2026 employees.csv.gz
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 employees.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ |
```

4. Create a zip archive containing

employees.csv and orders.csv.

Output: data_files.zip

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ zip data_files.zip employees.csv orders.csv
  adding: employees.csv (stored 0%)
  adding: orders.csv (stored 0%)
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ ls -lrth
total 0
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 summary.txt
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 orders.csv
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 notes.txt
-rwxrwxrwx 1 prashant prashant 34 Mar 18 2026 employees.csv.gz
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 employees.csv
-rwxrwxrwx 1 prashant prashant 324 Mar 18 2026 data_files.zip
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ |
```


5. List contents of the zip file.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ unzip -l data_files.zip
Archive:  data_files.zip
  Length      Date    Time    Name
-----
      0  2026-03-18  09:29    employees.csv
      0  2026-03-18  09:29    orders.csv
-----
      0                          2 files
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ |
```

6. Extract the zip file into a folder

called unzipped_data.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ unzip data_files.zip -d unzipped_data
Archive:  data_files.zip
  extracting: unzipped_data/employees.csv
  extracting: unzipped_data/orders.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ ls -rlth
total 0
-rwxrwxrwx 1 prashant prashant    0 Mar 18 09:29 summary.txt
-rwxrwxrwx 1 prashant prashant    0 Mar 18 09:29 orders.csv
-rwxrwxrwx 1 prashant prashant    0 Mar 18 09:29 notes.txt
-rwxrwxrwx 1 prashant prashant   34 Mar 18 09:29 employees.csv.gz
-rwxrwxrwx 1 prashant prashant    0 Mar 18 09:29 employees.csv
-rwxrwxrwx 1 prashant prashant  324 Mar 18 09:59 data_files.zip
drwxrwxrwx 1 prashant prashant  4.0K Mar 18 10:02 unzipped_data
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ |
```

Linux Task 3 - File Permissions (chmod)

First understand the permission format.

When you run: `ls -l`

You see something like:

```
-rw-r--r-- 1 user group 1024 Mar 8 employees.csv
```

The first 10 characters are permission bits:

- Position 1 : file type (- is file, d is dir)
- Positions 2-4 : owner permissions (rwx)
- Positions 5-7 : group permissions (rwx)
- Positions 8-10 : others permissions (rwx)

r = read (4)

w = write (2)

x = execute (1)

Now perform:

1. Check current permissions of employees.csv

using ls -l.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ ls -l employees.csv
-rwxrwxrwx 1 prashant prashant 0 Mar 18  2026 employees.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ chmod u-w employees.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ ls -l employees.csv
-rwxrwxrwx 1 prashant prashant 0 Mar 18  2026 employees.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/linux/project_files$ |
```